



SLOGAN

EMBEDDED SYSTEM

SPEED DETECTOR

DARI RAKYAT UNTUK RAKYAT

PRESENTED BY

GROUP 3

OUR TEAM

**Firmware
Developer &
Report**



Yohana indah

**Hardware
Assembly &
README**



Nayla Pramesti

**Firmware
Developer &
Simulation**



Gilardino

**Firmware
Developer &
Simulation**



Agib Pratama

Background & Problem Statement

Road Safety Crisis

Speeding is one of the leading causes of road accidents worldwide. In urban and semi-urban environments, vehicles frequently exceed speed limits, resulting in preventable fatalities and injuries that place a heavy burden on emergency services and public infrastructure.

Limitations of Manual Enforcement

Conventional speed enforcement depends on human officers stationed at fixed points, which is expensive, inconsistent, and only covers a small fraction of roads.

Embedded Systems as a Solution

Microcontroller-based systems like the Arduino Uno (ATmega328P) offer sufficient computational power and programmable I/O peripherals to build autonomous, low-cost, always-on speed detectors that can be deployed widely without ongoing human supervision.

Our Project Approach

This project simulates an ETLE (Electronic Traffic Law Enforcement) speed camera at Hotwheels scale using two FC-51 infrared sensors spaced 20 cm apart. The entire firmware is written in AVR Assembly, exercising all eight embedded systems course modules in a single integrated system.



Core Formula: $\text{Speed (km/h)} = (\text{Distance (m)} \div \text{Time (s)}) \times 3.6$

Sensors

Two FC-51 infrared modules are mounted exactly 20 cm apart on the test track. Each sensor outputs a LOW signal when an object breaks its IR beam, which is used to trigger hardware interrupts.

Timing

INT0 fires when the vehicle breaks IR Sensor 1, resetting and starting 16-bit Timer1. INT1 fires when the vehicle breaks IR Sensor 2, stopping the timer. The difference in TCNT1 ticks is the elapsed travel time.

Threshold

The user can adjust the overspeed limit in real time using a potentiometer connected to ADC channel A0. The 10-bit ADC reading is mapped linearly to a speed range of 20 km/h (minimum) up to 100 km/h (maximum).

Outputs

Three output states: Green LED + 'SAFE SPEED' (within limit), Yellow LED + 'TOO SLOW' (below 10 km/h), Red LED + Buzzer + 'OVER SPEED!' (above threshold). Speed also sent via USART to Serial Monitor.

Persistence

Every time an overspeed event occurs, the detected speed value is written to a dedicated EEPROM address. On next power-up, the system reads and displays this stored value on the LCD before entering idle mode.



GPIO

All I/O pins configured via DDRD/DDR B registers in assembly. IR sensors set as inputs (PD2, PD3); Red/Yellow/Green LEDs and buzzer set as outputs.



USART

USART0 initialized with baud rate divisor in UBRR0H/L. On each detection, the computed speed string is polled and sent character-by-character via UDR0.



Arithmetic

16-bit speed calculation performed entirely in AVR Assembly using MUL, ADD, ADIW instructions. Timer ticks are converted to milliseconds, then to km/h.



Timer

16-bit Timer1 in Normal mode measures elapsed time between sensor events. Prescaler selected to cover expected speed range. TCNT1H/L reset on INT0, read on INT1.



Ext. Interrupt

INT0 and INT1 configured in EICRA for falling-edge detection. ISR for INT0 starts Timer1; ISR for INT1 stops it and sets a flag for main-loop processing.



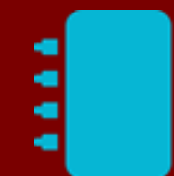
PWM + EEPROM

Timer0 Fast PWM on OC0A (PD6) drives the active buzzer during overspeed. Last overspeed speed value written to EEPROM using EEAR/EEDR/EECR registers.



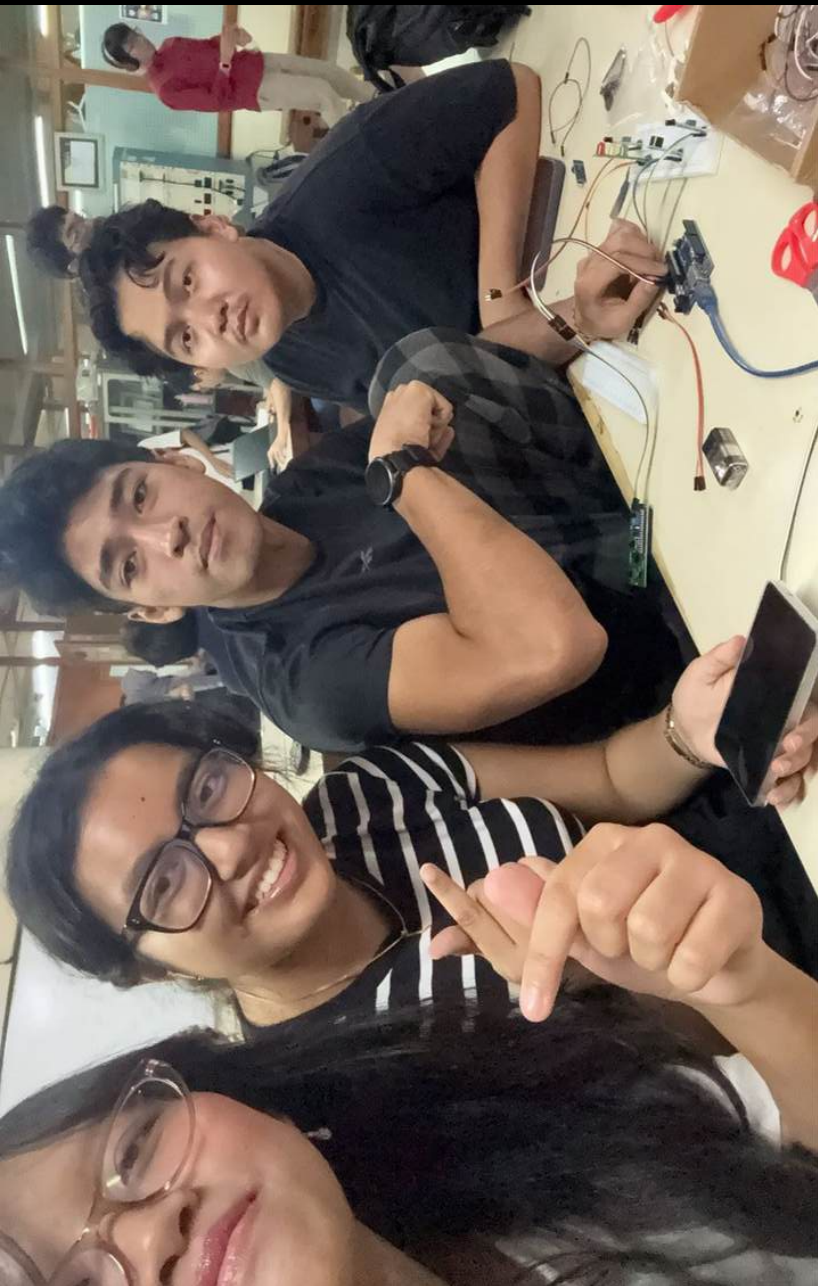
ADC

Timer0 Fast PWM on OC0A (PD6) drives the active buzzer during overspeed. Last overspeed speed value written to EEPROM using EEAR/EEDR/EECR registers.



I2C / TWI

TWI peripheral initialized via TWBR/TWCR. Low-level START, address, data, and STOP subroutines drive the PCF8574 I2C backpack of the 16x2 LCD display.



Arduino Uno (ATmega328P)

Main microcontroller running at 16 MHz. Provides the AVR core, 32 I/O pins, built-in Timer0/1, USART0, ADC, TWI, and 1 KB EEPROM. All firmware written in pure AVR Assembly — no Arduino libraries used.

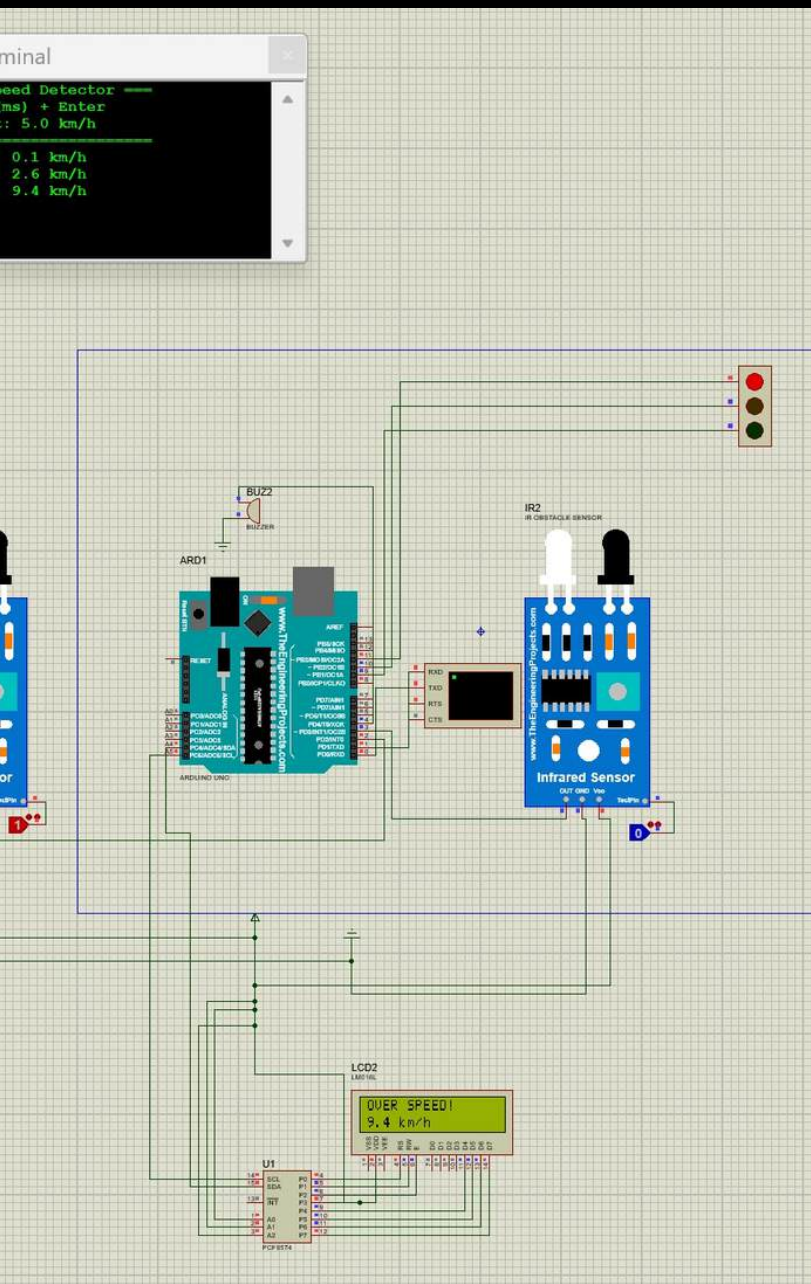
IR Sensor Module FC-51 x2

Reflective infrared proximity sensors. OUT pin goes LOW when an object passes through the beam. Sensor 1 OUT connected to D2 (INT0) and Sensor 2 OUT to D3 (INT1). Both powered from Arduino 5V/GND rails.

I2C LCD 16x2 (PCF8574)

16-character, 2-line alphanumeric display with a PCF8574 I2C I/O expander backpack. SDA connected to A4 and SCL to A5 — the dedicated TWI hardware pins of the ATmega328P.

Hardware Components & Wiring



Traffic Light LED Module

Three discrete LEDs (Red, Yellow, Green) in a traffic-light housing. Red connected to D11, Yellow to D10, Green to D9. Each LED indicates a different speed status state to the observer.

Active Buzzer Module

Emits an audible tone when powered. Connected to D8. Driven by Timer0 Fast PWM output on OC0A, generating a square wave whose duty cycle is controlled via the OCR0A register.

Potentiometer (10k ohm)

Wiper terminal connected to A0 (PC0, ADC channel 0). Rotating the knob produces a 0–5V analog voltage that the ADC converts to a 10-bit value, mapped to the 20–100 km/h threshold range.

Hardware Components & Wiring



Firmware Execution Flow – AVR Assembly

1

Initialize

Configure DDRD/DDR B for GPIO directions. Set UBRR0 for 9600 baud USART. Initialize Timer0/1 registers. Configure ADMUX + ADCSRA for ADC. Set TWBR for 100 kHz I2C. Enable INT0/INT1 in EIMSK. Read last overspeed value from EEPROM and display on LCD.

2

IDLE/STANDBY

System enters main polling loop. LCD displays 'Waiting...' and Yellow LED is turned ON to signal readiness. The ADC continuously samples the potentiometer to update the active speed threshold. CPU waits for INT0 to fire.

3

MEASURE TIME

INT0 ISR: TCNT1H and TCNT1L are cleared to zero, then Timer1 is started by writing a prescaler value to TCCR1B. INT1 ISR: Timer1 is halted by clearing TCCR1B. A 'speed_ready' flag is set to signal the main loop.

4

CALCULATE SPEED

Main loop reads TCNT1 tick count. Ticks are multiplied by the timer resolution (us/tick) to get elapsed microseconds, then converted to seconds. $\text{Speed} = (0.2 \text{ m} / \text{time}_s) \times 3.6$. Result stored in register pair as integer km/h.



Firmware Execution Flow – AVR Assembly

5

COMPARE & BRANCH

Latest ADC reading is scaled to km/h and used as the overspeed threshold. The calculated speed is compared against 10 km/h (too slow boundary) and against the ADC threshold using AVR CP/BRLO/BRSH branch instructions.

6

OUTPUT & LOG

Correct LED activated via GPIO. I2C LCD updated with speed value and status string. If overspeed: Timer0 PWM starts buzzer on OC0A, speed written to EEPROM. Speed string transmitted character-by-character via USART to PC.

Speed Classification & Output Logic

TOO SLOW

Condition	Detected speed is below 10 km/h the vehicle is moving too slowly to be measured accurately or is not fully crossing both sensors.
LED	Yellow LED activated on D10 to signal that a vehicle was detected but is moving beneath the minimum measurable speed range.
LCD Line 1	Displays 'TOO SLOW' prominently on the first row of the 16x2 I2C LCD display to alert the operator.
LCD Line 2	Shows the actual measured speed in km/h so the operator can see the exact reading that triggered this state.
Buzzer	Buzzer remains OFF. No overspeed violation has occurred; no EEPROM write is performed for this state.

SAFE SPEED

Condition	Detected speed falls between 10 km/h and the user-configured ADC threshold the vehicle is within the acceptable speed range.
LED	Green LED activated on D9. This signals normal, lawful travel speed. Yellow and Red LEDs are explicitly turned OFF.
LCD Line 1	Displays 'SAFE SPEED' on row 1 of the LCD to confirm to the operator that no violation has been detected.
LCD Line 2	The measured vehicle speed is printed in km/h on the second LCD line for reference and logging purposes.
Buzzer	Buzzer remains OFF. No violation logged. EEPROM not written. USART still transmits the speed value to the Serial Monitor.

OVERSPEED

Condition	Detected speed exceeds the potentiometer-defined threshold (range: 20-100 km/h). A speed violation has been registered by the system.
LED	Red LED activated on D11. Yellow and Green LEDs are turned OFF. The red LED stays ON for the duration of the alarm period.
LCD Line 1	Displays 'OVER SPEED!' in uppercase on the first row of the LCD to clearly indicate a violation event to the operator.
LCD Line 2	The exact speed that triggered the violation is shown in km/h, both on the LCD and simultaneously sent via USART to the PC.
Buzzer	Timer0 Fast PWM signal generated on OC0A (D6). Drives the active buzzer with a square wave. EEPROM writes the violation speed.

Software Simulation (Proteus)

Initialization :

LCD correctly displayed 'ETLE Detector Initializing...' on startup, confirming I2C TWI communication and LCD nibble protocol were functioning as intended.

Idle State:

System transitioned to standby mode, LCD showed 'Waiting...', and the Yellow LED illuminated on the traffic light module correctly indicating the system was ready for a vehicle.

Safe Speed Test:

Manual logic state switching produced a measured velocity of 0.3 km/h against the 5.0 km/h software threshold. LCD showed 'SAFE SPEED' and Green LED

Overspeed Test:

USART Virtual Terminal was used to inject a simulated 9.4 km/h reading, bypassing Proteus timing limitations. System correctly triggered 'OVER SPEED!', lit Red LED, and sounded buzzer.

EEPROM + USART::

Overspeed value was written to EEPROM and verified on next simulated power cycle. USART Virtual Terminal showed correct speed strings output in real-time for all test cases.

Physical Hardware Testing

IR Sensor Detection:

Both FC-51 sensors correctly detected the toy car passing through their beams and generated the expected falling-edge signals on D2 and D3 without missing any detection events.

I2C LCD Display:

Realtime speed values and status messages (Safe Speed / Over Speed) were accurately displayed on the 16x2 LCD via the I2C TWI interface, updating correctly after every detection event.

Threshold Adjustment:

Due to mechanical constraints, overspeed threshold was lowered to 1.0 km/h via the potentiometer. The ADC correctly mapped the knob position to the adjusted threshold in real time.

Overspeed Protocol:

When the taller test car exceeded 1.0 km/h, the system correctly activated the Red LED on D11, started the PWM buzzer on D8, and logged the speed to EEPROM all simultaneously.

All Modules Verified:

All 8 embedded modules (GPIO, USART, Arithmetic, Timer1, Interrupts, PWM, EEPROM, I2C) operated correctly and concurrently in physical hardware under real-world test conditions.

Analysis & Challenges



Hardware Timing Precision

The 16-bit Timer1, triggered immediately by external hardware interrupts INT0 and INT1, measured elapsed time between sensor events with high accuracy. Because the ISRs handle start/stop directly, there is zero CPU polling overhead and no software-induced timing jitter. This is a key advantage of using hardware peripherals rather than software delay loops.

I2C Pin Efficiency & Protocol

Using the PCF8574 I2C backpack for the LCD reduced the number of dedicated microcontroller pins from 7 (parallel LCD) to just 2 (SDA/SCL). The entire TWI communication — START, address, data byte, and STOP conditions — was implemented manually in AVR Assembly, demonstrating low-level protocol mastery without relying on any library abstraction.



Attention to detail

The FC-51 IR module emitter and receiver are mounted side-by-side on the PCB, requiring the diode legs to be bent 90 degrees to face across the track instead of forward. This manual bending introduced alignment inconsistency — sensors could not be guaranteed to sit at exactly 90 degrees — and risked permanently damaging the sensor pins. This limited which vehicles could reliably break the IR beam.

Communication skills

Standard low-profile Hotwheels cars could not clear the raised sensor beams. A taller Hotwheels car was required, but this vehicle was too large to fit the spring-loaded Hotwheels launcher, meaning it could only be pushed by hand. This fundamentally limited the achievable test speed, making it necessary to lower the overspeed threshold from 5 km/h (simulation) down to 1 km/h for hardware validation.



Future Improvements & Recommendations

Robust Mechanical Sensor Mounting

Design and 3D-print dedicated enclosures that hold the FC-51 IR diodes at a precise 90-degree angle relative to the track without requiring manual bending. Rigid mounts would guarantee consistent beam alignment across both sensors, enable detection of any standard vehicle profile, and eliminate the risk of pin damage during assembly or repositioning.

Upgrade to Laser Break-Beam or Ultrasonic Sensors

Replace the FC-51 reflective IR sensors with laser break-beam modules or HC-SR04 ultrasonic sensors. Laser break-beams provide a far longer and more reliable detection range, while ultrasonic sensors are immune to ambient light interference. Either choice would remove the mechanical bending constraint entirely and allow testing with any vehicle height.

Integrated Camera Module for Plate Capture

Add an OV7670 or similar UART/SPI camera module that is triggered automatically when an overspeed EEPROM write event occurs. This would allow the system to capture an image of the violating vehicle at the moment of detection, bringing the prototype substantially closer to real-world ETLE deployment behavior where photographic evidence is mandatory.

Wireless Telemetry via Wi-Fi or Bluetooth

Replace the wired USART-to-USB serial link with an ESP8266 Wi-Fi module or HC-05 Bluetooth module. Speed logs, violation timestamps, and EEPROM records could then be transmitted wirelessly to a central monitoring dashboard or mobile application, eliminating the need for physical cable connections and enabling remote multi-unit deployment.

Conclusion



All 8 embedded systems modules GPIO, USART, Arithmetic, Timer1, External Interrupts, PWM/EEPROM, ADC, and I2C/TWI were successfully designed, coded, and integrated in pure AVR Assembly language with no reliance on Arduino libraries or high-level abstractions.



Hardware interrupts (INT0 and INT1) combined with the 16-bit Timer1 peripheral delivered accurate, non-blocking elapsed-time measurement between the two IR sensors, correctly computing vehicle speed using the distance-over-time formula.



The ADC-based adjustable threshold proved the system's real-world flexibility: the speed limit was tuned from 5 km/h in Proteus simulation down to 1 km/h on physical hardware to compensate for mechanical constraints, and the system responded correctly in both configurations.



Both Proteus software simulation and physical hardware testing confirmed end-to-end correct operation of all output modes: LED traffic-light indicators, I2C LCD status display, PWM buzzer alarm, USART serial logging, and EEPROM overspeed violation persistence across power cycles.

Team Contributions & Division of Work

Yohana Indah N.

Firmware Developer & Report Author

- GPIO: configured all DDRD/DDR B registers, set pin directions for LEDs, IR inputs, and buzzer output
- USART: implemented UBRR0 initialization, TX polling via UCSRA, and character-by-character string transmission
- Arithmetic: wrote 16-bit Assembly routines to convert TCNT1 ticks to km/h using MUL, ADD, and ADIW
- Timer1: configured Normal mode with prescaler selection, managed TCNT1H/L reset and read sequences
- Authored the full PDF project report including all technical chapters and analysis sections

Nayla Pramesti A.

Hardware Assembly & Documentation

- Assembled the full breadboard circuit including all power rails, GPIO connections, and component wiring
- Mounted and positioned both FC-51 IR sensors on the physical track at the correct 20 cm spacing
- Integrated all hardware components and performed end-to-end physical testing of the assembled circuit
- Wrote the README documentation covering hardware setup, pin mapping, and usage instructions
- Operated the hardware demo during testing and validated all three speed status output states

Muhammad Naufal G.

Firmware Developer & Simulation

- External Interrupts: configured EICRA for falling-edge INT0/INT1, wrote ISR for Timer1 start/stop and flag-setting
- PWM: configured Timer0 Fast PWM mode using TCCR0A/B, set OCR0A duty cycle to drive active buzzer
- EEPROM: implemented read/write routines using EEAR, EEDR, and EECR registers for violation logging
- Built and maintained the full Proteus simulation schematic including ATmega328P, LCD, IR sensors, LEDs
- Used the USART Virtual Terminal in Proteus to simulate high-speed timing inputs and validate branching logic

Agib Pratama

Firmware Developer & Simulation

- ADC: configured ADMUX and ADCSRA in Assembly, polled ADIF flag, and mapped 10-bit result to 20-100 km/h
- I2C/TWI: initialized TWBR, wrote START/address/data/STOP subroutines, implemented PCF8574 nibble LCD protocol
- Wrote all higher-level LCD display functions built on top of the low-level TWI communication subroutines
- Contributed to Proteus simulation including LCD component wiring and PCF8574 I2C backpack configuration
- Validated ADC threshold mapping and confirmed LCD output accuracy across all three speed classification states

THANK YOU